

Universidad Técnica Estatal de Quevedo
Facultad de Ciencias de la Computación y Diseño Gráfico
Carrera de Ingeniería en Software

Sistema de Gestión de Solicitudes para Soporte Técnico de Internet

Entrega 2 — Diseño de Arquitectura de Microservicios

Versión 2.0

Propuesta de Proyecto para Aplicaciones Distribuidas

Arquitecto:	Pacheco Cárdenas Cristhian Daniel
Líder de Desarrollo:	Carpio Mendoza Carlos José
Responsable de Calidad:	Álvarez Párraga Jeremy Alexis
Responsable de Documentación:	Cando Moreno Robinson Rodrigo

Docente:	Gleiston Cicerón Guerrero Ulloa
Asignatura:	Aplicaciones Distribuidas
Período Académico:	Séptimo Semestre
Fecha de Entrega:	Junio 2026

Índice general

1. Registro de Cambios respecto a la Entrega 1	3
1.1. Línea Base: Resumen Ejecutivo de la Entrega 1	3
1.1.1. Problema que se resuelve	3
1.1.2. Usuarios del sistema	3
1.1.3. Un sistema distribuido es la solución apropiada	3
1.1.4. Alcance del período académico	4
1.2. Contexto de la Revisión	4
1.3. Cambios Arquitectónicos Adoptados	5
2. Domain-Driven Design y Bounded Contexts	6
2.1. Dominio de Negocio	6
2.1.1. Lenguaje Ubicuo (Ubiquitous Language)	6
2.2. Identificación de Bounded Contexts	6
2.2.1. BC-1: Autenticación y Autorización	7
2.2.2. BC-2: Gestión de Tickets	7
2.2.3. BC-3: Notificaciones	7
2.2.4. BC-4: Inteligencia Artificial	8
2.2.5. BC-5: Reportes y Análisis	8
3. Diseño de Microservicios	9
3.1. Principios de Descomposición	9
3.2. Catálogo de Microservicios	9
3.3. Descripción Detallada de Cada Microservicio	10
3.3.1. auth-service	10
3.3.2. ticket-service	11
3.3.3. notification-service	11
3.3.4. ai-service	12
3.3.5. report-service	12
4. Diagrama de Arquitectura Completa	13
4.1. Vista General del Sistema	13
4.2. Componentes de Infraestructura	14
5. Comunicaciones: REST Síncrono vs. Kafka Asíncrono	15
5.1. Criterios de Clasificación	15

5.2. Comunicaciones Síncronas — REST	16
5.3. Comunicaciones Asíncronas — Apache Kafka	17
5.3.1. Estructura de un Evento Kafka	17
6. Patrones de Microservicios Aplicados	19
6.1. Patrón 1: CQRS (Command Query Responsibility Segregation)	19
6.2. Patrón 2: Saga (Coreografía)	20
6.3. Patrón 3: Circuit Breaker	21
7. Contrato del API REST	23
7.1. Convenciones Generales	23
7.2. auth-service — Endpoints	23
7.3. ticket-service — Endpoints	24
7.3.1. Ejemplo de Request y Response	24
7.4. ai-service — Comunicación	25
7.5. report-service — Endpoints	25
8. Estrategia de Contenerización	26
8.1. Filosofía de Contenerización	26
8.2. Estructura de Imágenes Docker	26
8.3. Dockerfile de Referencia — ticket-service	26
8.4. Docker Compose — Orquestación Local	27
8.5. Variables de Entorno y Gestión de Secretos	30
8.6. Estrategia de Red en Docker	30
9. Conclusiones y Próximos Pasos	31
9.1. Logros de la Entrega 2	31
9.2. Plan para la Semana 7 — Implementación	31
Declaratoria de Uso de Inteligencia Artificial	33

Capítulo 1

Registro de Cambios respecto a la Entrega 1

Esta sección documenta las decisiones arquitectónicas que fueron ajustadas a partir de la retroalimentación recibida en la Entrega 1, así como las motivaciones técnicas que sustentaron cada cambio.

1.1 Línea Base: Resumen Ejecutivo de la Entrega 1

1.1.1 Problema que se resuelve

Muchas organizaciones, especialmente de tamaño pequeño y mediano, gestionan sus solicitudes de soporte técnico a través de canales informales como correos electrónicos, llamadas telefónicas o mensajería instantánea. Esta forma de trabajar genera desorganización, pérdida de información y tiempos de respuesta prolongados.

El presente proyecto propone un Sistema de Gestión de Solicitudes para Soporte Técnico de Internet, orientado a formalizar y estructurar el proceso de atención de incidencias relacionadas con la conectividad a Internet dentro de entornos organizacionales.

1.1.2 Usuarios del sistema

El sistema está dirigido a organizaciones que ofrecen o dependen de servicios de Internet y que necesitan un mecanismo ordenado para gestionar incidencias técnicas. Lo utilizarán tanto el cliente que reporta problemas como el equipo técnico encargado de resolverlos y el personal administrativo que supervisa y coordina el proceso de atención.

1.1.3 Un sistema distribuido es la solución apropiada

Si esta aplicación se construyera como un sistema único y centralizado, todo dependería de un solo punto: si ese punto falla, el servicio deja de funcionar por completo.

Un enfoque distribuido resuelve esto dividiendo la aplicación en partes separadas: la interfaz que ve el usuario, la lógica que procesa las solicitudes y la base de datos donde se guarda la información. Cada parte trabaja de forma independiente y se comunican entre sí. Esto trae beneficios directos:

- Si una parte falla, las demás pueden seguir funcionando.
- Si la cantidad de usuarios crece, se puede reforzar solo la parte que lo necesita.
- Si se requiere hacer un cambio, se modifica únicamente la parte correspondiente sin afectar el resto del sistema.

1.1.4 Alcance del período académico

Durante el período académico, el trabajo consistirá en tomar la aplicación web del Sistema de Gestión de Solicitudes para Soporte Técnico de Internet, ya existente, e identificar de qué manera puede reorganizarse bajo un enfoque distribuido. A partir de ese análisis, se irán aplicando gradualmente los conceptos trabajados a lo largo del semestre sobre su estructura y diseño: cómo se comunican sus componentes, cómo se organiza y distribuye su información, y de qué forma puede estructurarse en capas independientes.

La idea es transformar la aplicación web original en un ecosistema basado en microservicios desacoplados y orientados a eventos, dividiendo la plataforma en componentes individuales de autenticación, gestión de reportes, notificaciones e inteligencia artificial que interactuarán de forma que no esperarse el uno al otro para seguir trabajando.

1.2 Contexto de la Revisión

La Entrega 1 presentó el problema de negocio y justificó la adopción de un enfoque distribuido para el Sistema de Gestión de Solicitudes de Soporte Técnico de Internet. La retroalimentación del docente señaló la necesidad de profundizar en la descomposición funcional del sistema y de precisar los mecanismos de comunicación entre componentes.

1.3 Cambios Arquitectónicos Adoptados

Cuadro 1.1: Decisiones modificadas entre Entrega 1 y Entrega 2

Aspecto	Decisión en Entrega 1	Decisión en Entrega 2
Número de microservicios	No definido explícitamente	5 microservicios con fronteras claras por bounded context
Comunicación	Mencionada genéricamente	Clasificada: REST síncrono para operaciones CRUD críticas y Kafka asíncrono para eventos de dominio
Base de datos	Una sola base de datos compartida	Database-per-Service: cada microservicio gestiona su propia persistencia
Inteligencia artificial	Componente mencionado sin detalle	Microservicio independiente con API propia para clasificación y sugerencias
Orquestación	No considerada	Se incorpora API Gateway como punto único de entrada

Justificación General

El principio de *Database-per-Service* fue adoptado porque en la Entrega 1 se planteaba una base de datos compartida, lo que hubiera introducido acoplamiento implícito entre servicios y dificultado el escalado independiente. La separación de bases de datos permite evolucionar el esquema de datos de cada servicio sin impactar a los demás.

Capítulo 2

Domain-Driven Design y Bounded Contexts

2.1 Dominio de Negocio

El dominio central del sistema es la **gestión del ciclo de vida de solicitudes de soporte técnico de Internet**. Este dominio abarca desde el momento en que un cliente reporta una incidencia hasta la resolución confirmada y el cierre formal del ticket, pasando por la asignación a técnicos, las notificaciones de estado y el análisis inteligente de patrones de falla.

2.1.1 Lenguaje Ubicuo (Ubiquitous Language)

Cuadro 2.1: Glosario del Lenguaje Ubicuo del Dominio

Término	Definición en el dominio
Solicitud	Reporte formal de una incidencia técnica registrado por un Cliente.
Ticket	Representación interna de una Solicitud que posee un ciclo de vida trazable.
Incidencia	Evento de fallo o degradación de conectividad reportado.
Técnico	Agente responsable de diagnosticar y resolver una Solicitud.
Asignación	Acto de vincular un Ticket a un Técnico disponible.
Resolución	Acción que cierra una Solicitud con una solución documentada.
Escalamiento	Transferencia de un Ticket a un nivel superior de soporte.
Notificación	Mensaje enviado al Cliente o al Técnico ante cambios de estado.
SLA	Acuerdo de nivel de servicio que define tiempos máximos de atención.
Categoría	Clasificación automática o manual del tipo de incidencia.

2.2 Identificación de Bounded Contexts

Aplicando los principios de DDD, se identificaron **cinco bounded contexts** diferenciados por su responsabilidad y por el modelo de dominio que encapsulan.

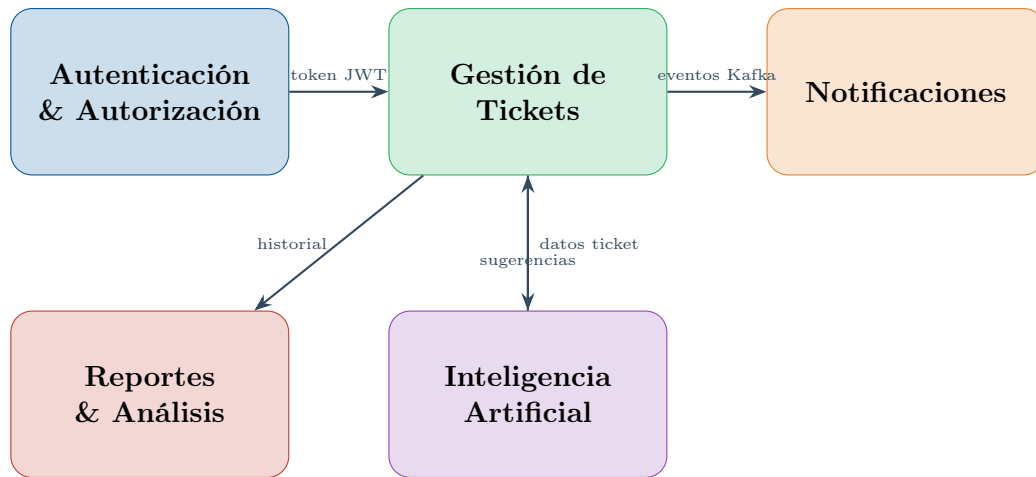


Figura 2.1: Context Map — Relaciones entre Bounded Contexts

2.2.1 BC-1: Autenticación y Autorización

Responsabilidad: Gestionar la identidad de todos los actores del sistema (Clientes, Técnicos y Administradores), emitir y validar tokens JWT, y controlar permisos basados en roles (RBAC).

Modelo de dominio clave: Usuario, Rol, Sesión, Token.

Relación con otros contextos: Proveedor de identidad para todos los demás bounded contexts. Actúa como *upstream* en el context map.

2.2.2 BC-2: Gestión de Tickets

Responsabilidad: Centralizar el ciclo de vida completo de una solicitud de soporte — creación, asignación, actualización de estado, escalamiento y cierre. Es el *core domain* del sistema.

Modelo de dominio clave: Ticket, Asignación, Comentario, Escalamiento, EstadoTicket.

Eventos de dominio emitidos: TicketCreado, TicketAsignado, TicketResuelto, TicketEscalado.

2.2.3 BC-3: Notificaciones

Responsabilidad: Consumir eventos del bus de mensajes y distribuir notificaciones a los destinatarios apropiados (email, SMS, push).

Modelo de dominio clave: Notificación, Canal, Plantilla, RegistroEnvío.

2.2.4 BC-4: Inteligencia Artificial

Responsabilidad: Analizar el contenido de nuevas solicitudes para categorizarlas automáticamente, estimar prioridad y sugerir técnicos disponibles basándose en patrones históricos.

Modelo de dominio clave: Predicción, Categoría, ModeloML, SugerenciaTécnico.

2.2.5 BC-5: Reportes y Análisis

Responsabilidad: Construir y exponer métricas operativas: tiempo promedio de resolución, cumplimiento de SLA, volumen de incidencias por período y por tipo.

Modelo de dominio clave: Reporte, MétricaSLA, Estadística.

Capítulo 3

Diseño de Microservicios

3.1 Principios de Descomposición

Cada bounded context se materializa en un microservicio independiente siguiendo los principios de **alta cohesión** y **bajo acoplamiento**. La regla *Single Responsibility* se aplica a nivel de servicio: cada microservicio modifica únicamente su propia base de datos y es el único propietario de sus datos.

3.2 Catálogo de Microservicios

Cuadro 3.1: Microservicios del sistema y sus características principales

Servicio	Puerto	BD	Responsabilidad Principal	Tecnología
auth-service	8001	PostgreSQL	Registro, login, emisión y validación JWT, RBAC	Spring Boot
ticket-service	8002	PostgreSQL	CRUD de tickets, estados, asignaciones, SLA	Spring Boot
notification-service	8003	MongoDB	Consumo de eventos, despacho multicanal	Node.js
ai-service	8004	MongoDB	Clasificación NLP vía Kafka, priorización, sugerencias	FastAPI (Python)
report-service	8005	PostgreSQL	Consultas CQRS de lectura, métricas, exportación	Spring Boot

3.3 Descripción Detallada de Cada Microservicio

3.3.1 auth-service

Microservicio: auth-service

Responsabilidades:

- Registrar nuevos usuarios con roles predefinidos (CLIENTE, TÉCNICO, ADMIN).
- Autenticar credenciales y emitir tokens JWT con *claims* de rol y permisos.
- Revocar sesiones y gestionar la rotación de tokens de actualización (*refresh tokens*).
- Exponer un endpoint de validación de token consumido por el API Gateway.

Base de datos: PostgreSQL — esquema `auth_db` con tablas `users`, `roles`, `user_roles`, `refresh_tokens`.

Comunicación: Solo síncrona (REST). No produce eventos de dominio propios.

Modelo de datos — auth-service

```
1  -- Tabla principal de usuarios
2  CREATE TABLE users (
3      id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
4      email       VARCHAR(255) UNIQUE NOT NULL,
5      password    VARCHAR(255) NOT NULL, -- bcrypt hash
6      full_name   VARCHAR(255),
7      is_active   BOOLEAN DEFAULT TRUE,
8      created_at  TIMESTAMP DEFAULT NOW()
9  );
10
11 -- Roles del sistema
12 CREATE TABLE roles (
13     id    SERIAL PRIMARY KEY,
14     name VARCHAR(50) UNIQUE NOT NULL -- CLIENTE, TECNICO, ADMIN
15 );
16
17 -- Tokens de refresco
18 CREATE TABLE refresh_tokens (
19     id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
20     user_id     UUID REFERENCES users(id) ON DELETE CASCADE,
21     token       TEXT NOT NULL,
22     expires_at  TIMESTAMP NOT NULL,
23     revoked     BOOLEAN DEFAULT FALSE
24 );
```

Listing 3.1: Esquema simplificado de auth_db

3.3.2 ticket-service

Microservicio: ticket-service

Responsabilidades:

- Crear, actualizar y consultar tickets de soporte técnico.
- Gestionar el ciclo de vida del ticket a través de una máquina de estados.
- Asignar tickets a técnicos disponibles manualmente o por sugerencia de ai-service.
- Controlar el SLA y emitir alertas de incumplimiento.
- Publicar eventos de dominio en Kafka ante cada transición de estado.

Base de datos: PostgreSQL — esquema `ticket_db`.

Comunicación: REST para operaciones CRUD; Kafka para publicar eventos.

Máquina de Estados del Ticket

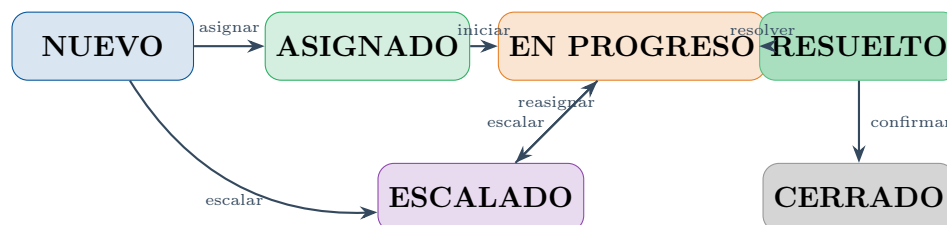


Figura 3.1: Máquina de estados del Ticket en ticket-service

3.3.3 notification-service

Microservicio: notification-service

Responsabilidades:

- Suscribirse a los topics Kafka del `ticket-service`.
- Seleccionar el canal de envío apropiado (email, SMS, push) según las preferencias del destinatario.
- Aplicar plantillas de mensaje parametrizadas.
- Registrar el historial de envíos y gestionar reintentos en caso de fallo.

Base de datos: MongoDB — colección `notifications` (documentos flexibles para distintos canales).

Comunicación: Solo asíncrona (consumidor Kafka). No expone REST público.

3.3.4 ai-service

Microservicio: ai-service

Responsabilidades:

- Analizar la descripción textual de un nuevo ticket mediante procesamiento de lenguaje natural.
- Clasificar automáticamente la categoría (DNS, velocidad, hardware, configuración, etc.).
- Estimar el nivel de prioridad (CRÍTICO, ALTO, MEDIO, BAJO).
- Sugerir el técnico más adecuado basándose en historial y especialización.

Base de datos: MongoDB — almacena resultados de predicciones y datos de entrenamiento.

Comunicación: Solo asíncrona. Consume el evento `ticket.created` desde Kafka, procesa la clasificación y publica `ticket.classified` de vuelta al bus. No expone REST público.

3.3.5 report-service

Microservicio: report-service

Responsabilidades:

- Mantener un modelo de datos desnormalizado optimizado para lectura (patrón CQRS).
- Calcular y exponer KPIs operativos: tiempo medio de resolución, tasa de cumplimiento SLA, volumen por período.
- Generar reportes exportables en CSV y PDF.

Base de datos: PostgreSQL — esquema `report_db` con vistas materializadas.

Comunicación: Suscriptor Kafka para actualizar el modelo de lectura; REST para exponer consultas.

Capítulo 4

Diagrama de Arquitectura Completa

4.1 Vista General del Sistema

La arquitectura sigue el patrón de **microservicios con API Gateway**, donde todos los clientes externos se comunican exclusivamente a través del gateway. El bus de mensajes (Apache Kafka) desacopla los servicios que producen eventos de los que los consumen.

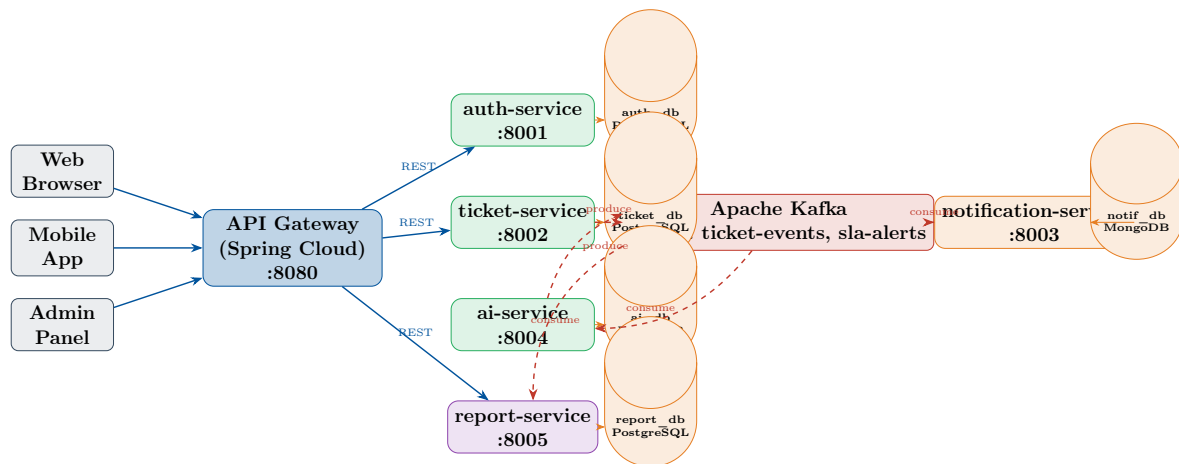


Figura 4.1: Diagrama de Arquitectura Completa del Sistema Distribuido

4.2 Componentes de Infraestructura

Cuadro 4.1: Componentes de infraestructura del sistema

Componente	Tecnología	Rol en la Arquitectura
API Gateway	Spring Cloud Gateway	Punto único de entrada, enrutamiento, rate limiting, autenticación delegada.
Bus de mensajes	Apache Kafka	Transporte asíncrono de eventos de dominio entre microservicios.
Service Registry	Eureka Server	Registro y descubrimiento de instancias de microservicios.
Config Server	Spring Cloud Config	Centralización de configuraciones por entorno (dev, staging, prod).
Contenerización	Docker + Compose	Empaquetado y orquestación local de todos los servicios.

Capítulo 5

Comunicaciones: REST Síncrono vs. Kafka Asíncrono

5.1 Criterios de Clasificación

La elección entre comunicación síncrona y asíncrona se realizó con base en dos criterios:

1. **Urgencia de respuesta:** Las operaciones que requieren respuesta inmediata para continuar el flujo (login, validación de token) usan REST síncrono.
2. **Desacoplamiento temporal:** Las operaciones donde el emisor no debe bloquear su hilo esperando al receptor (clasificación IA, notificaciones, actualización de reportes) usan Kafka asíncrono.

Decisión arquitectónica clave: ticket-service → ai-service

La comunicación entre `ticket-service` y `ai-service` es **exclusivamente asíncrona mediante Kafka**. Al crear un ticket, `ticket-service` lo persiste en estado `PENDIENTE_CLASIFICACION` y publica el evento `ticket.created`. El `ai-service` lo consume, clasifica y publica `ticket.classified`. El `ticket-service` actualiza entonces la categoría y prioridad. Esta decisión elimina el acoplamiento temporal entre ambos servicios y permite que el cliente reciba inmediatamente el ID del ticket, sin esperar a la IA. El patrón **Saga por coreografía** (Capítulo 6.2) implementa exactamente este flujo. Por tanto, **no existe ninguna llamada REST síncrona entre ticket-service y ai-service**.

5.2 Comunicaciones Síncronas — REST

Cuadro 5.1: Comunicaciones síncronas mediante REST/HTTP

Origen	Destino	Endpoint	Motivo de sincronía
API Gateway	auth-service	POST /auth/login	El cliente necesita el JWT antes de cualquier otra operación.
API Gateway	auth-service	GET /auth/validate	El gateway valida el token antes de enrutar la petición; debe ser síncrono.
API Gateway	ticket-service	POST /tickets	El cliente espera el ID del ticket (estado PENDIENTE_CLASIFICACION).
API Gateway	report-service	GET /reports/{id}	Consulta bajo demanda de métricas, requiere respuesta inmediata.

5.3 Comunicaciones Asíncronas — Apache Kafka

Cuadro 5.2: Comunicaciones asíncronas mediante Apache Kafka

Productor	Topic Kafka	Consumidores	Motivo de asincronía
ticket-service	ticket.created	ai-service, notification-service	La IA clasifica en segundo plano; el cliente ya recibió el ID del ticket.
ai-service	ticket.classified	ticket-service	ticket-service actualiza categoría/prioridad cuando la IA termina; no bloquea.
ticket-service	ticket.ready	notification-service	La notificación de confirmación al cliente se envía tras la clasificación.
ticket-service	ticket.assigned	notification-service	El técnico es avisado sin que ningún servicio espere confirmación.
ticket-service	ticket.resolved	notification-service, report-service	La actualización de reportes es eventual; no bloquea operaciones en curso.
ticket-service	ticket.escalated	notification-service	Notifica a varios actores; el desacoplamiento evita fallos en cascada.
ticket-service	sla.breached	notification-service, report-service	Las alertas de SLA son reactivas y no requieren respuesta inmediata.

5.3.1 Estructura de un Evento Kafka

```

1  {
2    "eventId": "evt-9f3a-4bc2-8e1d",
3    "eventType": "TICKET_CREATED",
4    "occurredAt": "2026-06-03T14:30:00Z",
5    "aggregateId": "tkt-00142",
6    "aggregateType": "Ticket",
7    "payload": {
8      "ticketId": "tkt-00142",
9      "clientId": "usr-00089",
10     "title": "Sin conexión a Internet",
11     "category": "CONECTIVIDAD",
12     "priority": "ALTO",
13     "slaDeadline": "2026-06-03T18:30:00Z"
14   },
15   "metadata": {

```

```
16     "source": "ticket-service",  
17     "version": "1.0"  
18   }  
19 }
```

Listing 5.1: Ejemplo de evento ticket.created publicado en Kafka

Capítulo 6

Patrones de Microservicios Aplicados

Se identificaron y diseñaron tres patrones de microservicios que resolverán desafíos concretos del sistema.

6.1 Patrón 1: CQRS (Command Query Responsibility Segregation)

CQRS — Separación de Comandos y Consultas

Problema que resuelve: El `ticket-service` recibe un alto volumen de escrituras (creación, actualización de estados) pero el `report-service` necesita consultas complejas con JOINS y agregaciones que penalizarían la base de datos operacional.

Solución aplicada: Se separa el modelo de escritura (Command) del modelo de lectura (Query). El `ticket-service` actualiza su propia base de datos (escritura). El `report-service` mantiene una base de datos de lectura desnormalizada que se actualiza escuchando los eventos Kafka del `ticket-service`.

Beneficio concreto: Las consultas de reportes no compiten con las escrituras operativas. El modelo de lectura puede optimizarse independientemente (índices, vistas materializadas).

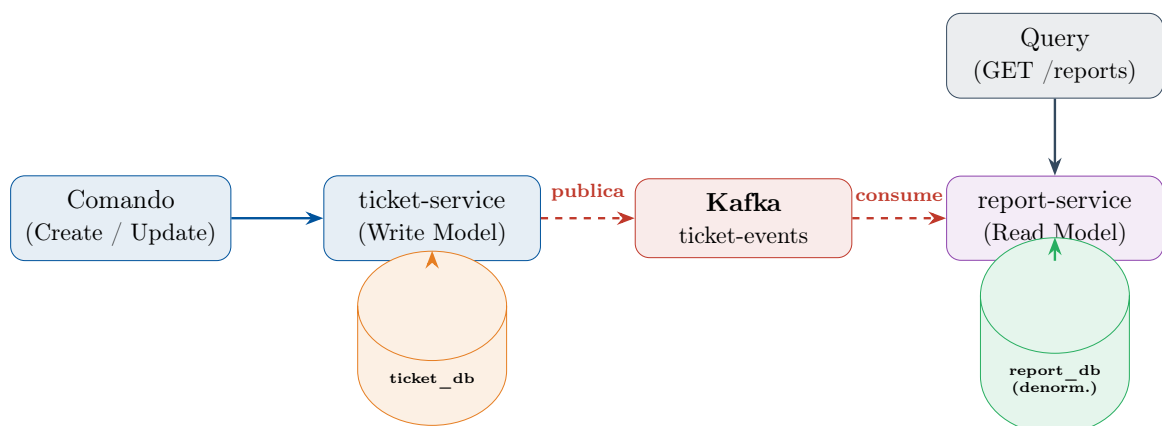


Figura 6.1: Implementación del patrón CQRS con espaciado y legibilidad optimizados

6.2 Patrón 2: Saga (Coreografía)

Saga — Transacciones Distribuidas por Coreografía

Problema que resuelve: La creación de un ticket involucra múltiples servicios: `ticket-service` crea el registro, `ai-service` lo clasifica y `notification-service` avisa al cliente. Si algún paso falla, el sistema debe compensar las acciones previas para mantener consistencia.

Solución aplicada: Implementar el patrón Saga con coreografía (sin orquestador central). Cada servicio reacciona al evento anterior y publica el siguiente. Si un paso falla, publica un evento de compensación.

Saga: Creación de Ticket

1. `ticket-service` crea el ticket en estado `PENDIENTE` → publica `ticket.created`.
2. `ai-service` consume `ticket.created`, clasifica → publica `ticket.classified`.
3. `ticket-service` consume `ticket.classified`, actualiza categoría y prioridad → publica `ticket.ready`.
4. `notification-service` consume `ticket.ready`, envía confirmación al cliente.

Compensación: Si la clasificación de IA falla, `ai-service` publica `ticket.classification.failed` y `ticket-service` elimina el ticket o lo deja en estado `ERROR`.

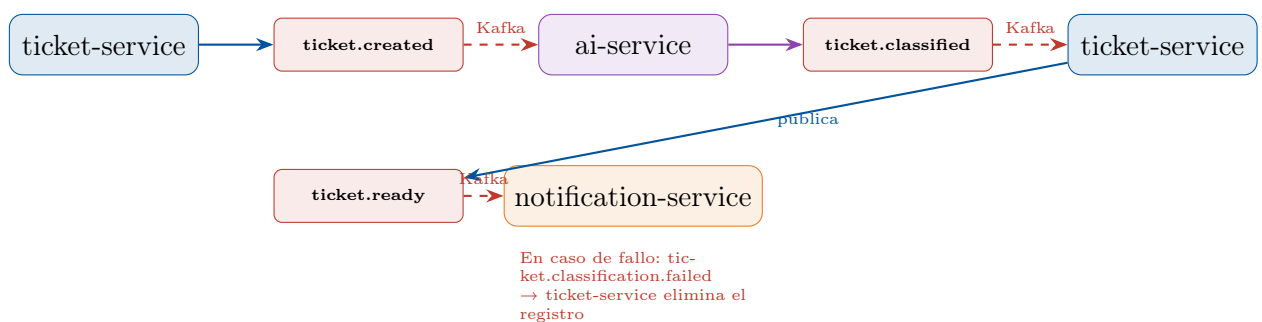


Figura 6.2: Saga de Creación de Ticket por Coreografía

6.3 Patrón 3: Circuit Breaker

Circuit Breaker — Resiliencia en el API Gateway

Aclaración arquitectónica: Dado que la comunicación entre `ticket-service` y `ai-service` es **asíncrona mediante Kafka** (patrón Saga), no existe una llamada HTTP bloqueante entre ellos y, por tanto, el Circuit Breaker **no aplica** en ese punto. El lugar correcto donde sí existe una llamada síncrona en riesgo de fallo en cascada es entre el **API Gateway** y los microservicios backend, particularmente `ticket-service`, que es el servicio más crítico del sistema.

Problema que resuelve: Si `ticket-service` se degrada (alta latencia, sobrecarga), el API Gateway acumularía peticiones en espera, agotaría su pool de hilos y dejaría de responder a **todos** los clientes, incluso a los que solo intentan hacer login. Esto es un fallo en cascada desde el servicio central hacia el gateway.

Solución aplicada: Implementar un Circuit Breaker (con Resilience4j) en el **API Gateway** para cada ruta hacia los microservicios. Tres estados:

- **CLOSED (Cerrado):** Las peticiones se enrutan normalmente al microservicio destino.
- **OPEN (Abierto):** Tras superar el umbral de fallos, el circuito abre. El gateway devuelve inmediatamente una respuesta `503 Service Unavailable` con un mensaje descriptivo, sin intentar contactar al microservicio. Los demás microservicios siguen funcionando.
- **HALF-OPEN (Semiabierto):** Pasado el tiempo de espera, se permite un número limitado de peticiones de prueba para verificar la recuperación.

Fallback concreto para ticket-service: El gateway responde `{.error": ".El servicio de tickets no está disponible temporalmente. Intente en unos minutos.", retryAfter": 30}` sin propagar el error al cliente como un timeout indefinido.

```
1 # application.yml          api-gateway (Spring Cloud Gateway)
2 spring:
3   cloud:
4     gateway:
5       routes:
6         - id: ticket-service-route
7           uri: http://ticket-service:8002
8           predicates:
9             - Path=/api/v1/tickets/**
10          filters:
11            - name: CircuitBreaker
12              args:
13                name: ticketServiceCB
```

```
14         fallbackUri: forward:/fallback/ticket-service
15
16 resilience4j:
17     circuitbreaker:
18         instances:
19             ticketServiceCB:
20                 registerHealthIndicator: true
21                 slidingWindowSize: 10           # Evalua ultimas 10
22                 peticiones                      # Minimo antes de calcular
23                 minimumNumberOfCalls: 5         # Abre si el 50% de las
24                 tasa de fallo                  llamadas fallan
25                 failureRateThreshold: 50
26                 waitDurationInOpenState: 15s    # Espera 15s en estado OPEN
27                 permittedNumberOfCallsInHalfOpenState: 3
28                 automaticTransitionFromOpenToHalfOpenEnabled: true
29
30 # Controlador de fallback en el gateway
31 # @RestController
32 # public class FallbackController {
33 #     @GetMapping("/fallback/ticket-service")
34 #     public ResponseEntity<Map<String, Object>> ticketFallback() {
35 #         return ResponseEntity.status(503).body(Map.of(
36 #             "error", "Servicio de tickets no disponible temporalmente",
37 #             "retryAfter", 30
38 #         ));
39 #     }
40 # }
```

Listing 6.1: Circuit Breaker en API Gateway con Resilience4j (application.yml)

Capítulo 7

Contrato del API REST

7.1 Convenciones Generales

- Todas las rutas tienen el prefijo `/api/v1/`.
- Autenticación mediante cabecera `Authorization: Bearer {jwt}`.
- Las respuestas siguen el estándar HTTP: 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 404 Not Found, 500 Internal Server Error.
- El cuerpo de las respuestas usa formato JSON con la envoltura: `{"data": ..., "message": ..., "timestamp": ...}`.

7.2 auth-service — Endpoints

Cuadro 7.1: Contrato REST — auth-service

Método	Ruta	Descripción	Roles
POST	<code>/api/v1/auth/register</code>	Registrar nuevo usuario	Público
POST	<code>/api/v1/auth/login</code>	Autenticar y obtener JWT	Público
POST	<code>/api/v1/auth/refresh</code>	Renovar access token	Autenticado
POST	<code>/api/v1/auth/logout</code>	Revocar refresh token	Autenticado
GET	<code>/api/v1/auth/validate</code>	Validar token (uso interno del gateway)	Interno

7.3 ticket-service — Endpoints

Cuadro 7.2: Contrato REST — ticket-service

Método	Ruta	Descripción	Roles
POST	/api/v1/tickets	Crear nuevo ticket	CLIENTE
GET	/api/v1/tickets	Listar tickets (filtrado por rol)	ALL
GET	/api/v1/tickets/{id}	Obtener detalle de un ticket	ALL
PATCH	/api/v1/tickets/{id}/status	Cambiar estado del ticket	TÉCNICO, ADMIN
POST	/api/v1/tickets/{id}/assign	Asignar ticket a técnico	ADMIN
POST	/api/v1/tickets/{id}/comment	Agregar comentario al ticket	ALL
POST	/api/v1/tickets/{id}/escalate	Escalar ticket	TÉCNICO, ADMIN

7.3.1 Ejemplo de Request y Response

```

1 // REQUEST
2 POST /api/v1/tickets
3 Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...
4 Content-Type: application/json
5
6 {
7   "title": "Sin acceso a Internet desde las 08:00",
8   "description": "El router muestra luz roja. Ya se reinicio el
9     equipo.",
10  "contactPhone": "0991234567",
11  "address": "Av. Quevedo 123, Los Rios"
12 }
13
14 // RESPONSE 201 Created
15 {
16   "data": {
17     "ticketId": "tkt-00142",
18     "status": "NUEVO",
19     "category": "CONECTIVIDAD", // asignado por ai-service
20     "priority": "ALTO", // asignado por ai-service
21     "slaDeadline": "2026-06-03T18:30:00Z",
22     "assignedTechnician": null
23   },
24   "message": "Ticket creado exitosamente",
25   "timestamp": "2026-06-03T14:30:00Z"
26 }
```

Listing 7.1: POST /api/v1/tickets — Crear Ticket

7.4 ai-service — Comunicación

El **ai-service** **no expone endpoints REST públicos**. Su comunicación es exclusivamente asíncrona: consume el topic `ticket.created` desde Kafka, ejecuta la clasificación NLP y publica el resultado en el topic `ticket.classified`. Los administradores pueden consultar predicciones históricas a través del **report-service**.

Cuadro 7.3: Flujo de mensajes del ai-service

Dirección	Topic Kafka	Acción	Descripción
Consume	<code>ticket.created</code>	Clasificar	Recibe el texto del ticket y ejecuta el modelo NLP.
Produce	<code>ticket.classified</code>	Notificar resultado	Publica categoría, prioridad y sugerencia de técnico.
Produce	<code>ticket.classified.failed</code>	Compañía de Saga	Publica si el modelo falla; ticket-service deja el ticket en estado ERROR.

7.5 report-service — Endpoints

Cuadro 7.4: Contrato REST — report-service

Método	Ruta	Descripción	Roles
GET	/api/v1/reports/sla	Métricas de cumplimiento de SLA	ADMIN
GET	/api/v1/reports/volume	Volumen de tickets por período	ADMIN
GET	/api/v1/reports/technicians	Rendimiento por técnico	ADMIN
GET	/api/v1/reports/export	Exportar reporte en CSV o PDF	ADMIN

Capítulo 8

Estrategia de Contenerización

8.1 Filosofía de Contenerización

Cada microservicio se empaqueta en su propia imagen Docker, lo que garantiza aislamiento de dependencias y reproducibilidad de entornos. La estrategia sigue el principio *Build Once, Run Anywhere*: la misma imagen que se prueba localmente es la que se despliega en producción.

8.2 Estructura de Imágenes Docker

Cuadro 8.1: Imágenes Docker por microservicio

Servicio	Imagen Base	Tamaño aprox.	Notas
auth-service	eclipse-temurin:17-jre	~200 MB	Build con Maven multi-stage
ticket-service	eclipse-temurin:17-jre	~210 MB	Build con Maven multi-stage
notification-service	node:20-alpine	~90 MB	Alpine para reducir tamaño
ai-service	python:3.11-slim	~180 MB	Slim para minimizar superficie
report-service	eclipse-temurin:17-jre	~195 MB	Build con Maven multi-stage
api-gateway	eclipse-temurin:17-jre	~180 MB	Spring Cloud Gateway

8.3 Dockerfile de Referencia — ticket-service

```
1 #           Etapa 1: Compilacion

2 FROM maven:3.9-eclipse-temurin-17 AS builder
3 WORKDIR /app
4
5 # Copiar archivos de dependencias primero (caching de capas)
6 COPY pom.xml .
7 RUN mvn dependency:go-offline -B
8
9 COPY src ./src
10 RUN mvn clean package -DskipTests
11
12 #           Etapa 2: Imagen de produccion (JRE solamente)

13 FROM eclipse-temurin:17-jre-alpine
14 WORKDIR /app
15
16 # Usuario no-root para seguridad
17 RUN addgroup -S appgroup && adduser -S appuser -G appgroup
18 USER appuser
19
20 COPY --from=builder /app/target/ticket-service-*.jar app.jar
21
22 # Puerto de escucha
23 EXPOSE 8002
24
25 # Variables de entorno configurables
26 ENV SPRING_PROFILES_ACTIVE=docker
27 ENV JAVA_OPTS="-Xms256m -Xmx512m"
28
29 HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
30     CMD wget --quiet --tries=1 --spider http://localhost:8002/
31     actuator/health || exit 1
32
33 ENTRYPOINT ["sh", "-c", "java $JAVA_OPTS -jar app.jar"]
```

Listing 8.1: Dockerfile multi-stage para ticket-service (Spring Boot)

8.4 Docker Compose — Orquestación Local

```
1 version: '3.9'
2
```

```
3 networks:
4   soporte-net:
5     driver: bridge
6
7 volumes:
8   auth-db-data:
9   ticket-db-data:
10  report-db-data:
11  ai-db-data:
12  notif-db-data:
13  kafka-data:
14
15 services:
16
17   #           Infraestructura
18
19   zookeeper:
20     image: confluentinc/cp-zookeeper:7.5.0
21     environment:
22       ZOOKEEPER_CLIENT_PORT: 2181
23     networks: [soporte-net]
24
25   kafka:
26     image: confluentinc/cp-kafka:7.5.0
27     depends_on: [zookeeper]
28     ports: ["9092:9092"]
29     environment:
30       KAFKA_BROKER_ID: 1
31       KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
32       KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092
33       KAFKA_AUTO_CREATE_TOPICS_ENABLE: "true"
34     networks: [soporte-net]
35
36   api-gateway:
37     build: ./api-gateway
38     ports: ["8080:8080"]
39     depends_on: [auth-service, ticket-service]
40     environment:
41       AUTH_SERVICE_URL: http://auth-service:8001
42       TICKET_SERVICE_URL: http://ticket-service:8002
43     networks: [soporte-net]
44
45   #           Microservicios
46
47   auth-service:
```

```
46     build: ./auth-service
47     ports: ["8001:8001"]
48     environment:
49         DB_URL: jdbc:postgresql://auth-db:5432/auth_db
50         DB_USER: authuser
51         DB_PASS: authpass
52         JWT_SECRET: ${JWT_SECRET}
53     depends_on: [auth-db]
54     networks: [soporte-net]
55
56     ticket-service:
57         build: ./ticket-service
58         ports: ["8002:8002"]
59         environment:
60             DB_URL: jdbc:postgresql://ticket-db:5432/ticket_db
61             KAFKA_BOOTSTRAP: kafka:9092
62             AI_SERVICE_URL: http://ai-service:8004
63         depends_on: [ticket-db, kafka, auth-service]
64         networks: [soporte-net]
65
66     ai-service:
67         build: ./ai-service
68         ports: ["8004:8004"]
69         environment:
70             MONGO_URI: mongodb://ai-db:27017/ai_db
71         depends_on: [ai-db]
72         networks: [soporte-net]
73
74     #           Bases de datos
75
76     auth-db:
77         image: postgres:15-alpine
78         environment:
79             POSTGRES_DB: auth_db
80             POSTGRES_USER: authuser
81             POSTGRES_PASSWORD: authpass
82         volumes: [auth-db-data:/var/lib/postgresql/data]
83         networks: [soporte-net]
84
85     ticket-db:
86         image: postgres:15-alpine
87         environment:
88             POSTGRES_DB: ticket_db
89             POSTGRES_USER: ticketuser
90             POSTGRES_PASSWORD: ticketpass
91         volumes: [ticket-db-data:/var/lib/postgresql/data]
```

```

91     networks: [soporte-net]
92
93     ai-db:
94         image: mongo:7.0
95         volumes: [ai-db-data:/data/db]
96         networks: [soporte-net]

```

Listing 8.2: docker-compose.yml (fragmento representativo)

8.5 Variables de Entorno y Gestión de Secretos

Gestión de secretos en Docker Compose

Para el entorno de desarrollo local, las variables sensibles (contraseñas de base de datos, secreto JWT) se almacenan en un archivo `.env` que **no se incluye en el repositorio Git** (añadido a `.gitignore`). El archivo `.env.example` documenta las variables requeridas sin revelar valores reales.

```

JWT_SECRET=                clave mínima de 256 bits
POSTGRES_PASSWORD=         contraseña de cada base de datos
MONGO_INITDB_ROOT_PASSWORD= contraseña de MongoDB

```

8.6 Estrategia de Red en Docker

Todos los contenedores comparten la red `soporte-net` de tipo bridge. Solo los puertos necesarios para acceso externo (gateway, herramientas de administración) se exponen al host. Los microservicios se comunican entre sí mediante el nombre del servicio Docker como hostname, lo que elimina la necesidad de IPs estáticas.

Cuadro 8.2: Puertos expuestos al host vs. puertos internos

Servicio	Puerto interno	Puerto host	Acceso
api-gateway	8080	8080	Público (clientes)
auth-service	8001	8001	Solo desarrollo
ticket-service	8002	8002	Solo desarrollo
ai-service	8004	8004	Solo desarrollo
report-service	8005	8005	Solo desarrollo
notification-service	8003	—	Solo red interna
kafka	9092	9092	Solo desarrollo

Capítulo 9

Conclusiones y Próximos Pasos

9.1 Logros de la Entrega 2

Esta entrega consolidó el diseño arquitectónico del sistema con los siguientes resultados:

1. Se delimitaron **cinco bounded contexts** mediante DDD, cada uno con su lenguaje ubicuo propio.
2. Se diseñaron **cinco microservicios independientes**, cada uno con su propia base de datos, eliminando el acoplamiento de datos de la propuesta inicial.
3. Se clasificaron todas las comunicaciones entre servicios en **síncronas (REST)** para operaciones críticas y **asíncronas (Kafka)** para eventos de dominio.
4. Se documentaron tres patrones de microservicios (**CQRS, Saga, Circuit Breaker**) con implementaciones concretas adaptadas al dominio del sistema.
5. Se definió una estrategia de **contenerización Docker** completa, reproducible y lista para implementación.

9.2 Plan para la Semana 7 — Implementación

Cuadro 9.1: Tareas de implementación asignadas por rol

Tarea	Responsable	Descripción
Levantar infraestructura	Arquitecto	Docker Compose con Kafka, ZooKeeper y bases de datos.
auth-service + JWT	Líder de Desarrollo	Implementación de registro, login y validación de tokens.
ticket-service + Kafka	Líder de Desarrollo	CRUD de tickets y publicación de eventos.
ai-service (mock)	Resp. Calidad	Endpoint de clasificación con respuestas simuladas.
notification-service	Resp. Documentación	Consumer Kafka + envío de email con plantillas.
Pruebas de integración	Resp. Calidad	Pruebas end-to-end del flujo completo de creación de ticket.

Riesgos Identificados para la Implementación

- **Latencia del Circuit Breaker:** Si los umbrales configurados son demasiado estrictos, el circuito se abrirá innecesariamente. Se iniciará con una tasa de fallo del 50 % y se ajustará con datos reales.
- **Consistencia eventual en Saga:** El estado del ticket puede ser temporalmente inconsistente entre servicios. Se documentarán los tiempos máximos de propagación esperados.
- **Tamaño de imágenes Docker:** Las imágenes Python pueden crecer con las dependencias de ML. Se usarán builds multi-stage y `.dockerignore` para controlar el tamaño.

Declaratoria de Uso de Inteligencia Artificial

En cumplimiento de las normativas de honestidad académica y las políticas institucionales orientadas a la innovación tecnológica de la Universidad Técnica Estatal de Quevedo, los autores declaran haber utilizado herramientas de Inteligencia Artificial Generativa de manera asistida durante el desarrollo de esta entrega técnica.

El uso de estas tecnologías se acotó estrictamente a las siguientes actividades de soporte de ingeniería:

- **Auditoría de sintaxis e infraestructura:** Verificación estructural de los manifiestos de orquestación de contenedores (*Docker Compose*) y optimización multi-stage en los archivos de construcción (*Dockerfiles*).
- **Auditoría arquitectónica:** Identificación preliminar de redundancias en las fronteras conceptuales del diseño y consistencia semántica en las llamadas síncronas/asíncronas descritas.
- **Refinamiento estilístico y tipográfico:** Corrección ortográfica, cohesión de la documentación técnica y formateo avanzado del código fuente del informe en lenguaje tipográfico L^AT_EX.